

C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural
Networks

David Gonzalez¹, Joshua T. Guerin, Ph.D.²

Mathematical Sciences

University of Northern Colorado

Greeley, CO 80639

¹*****@bears.unco.edu

²*****@unco.edu

Abstract

This is the short abstract.

1 Methodology

1.1 Constructing the Logical Network

The bulk of our time on this project was spent on constructing the framework for logical networks, and testing it on an 8-bit adder. We chose this example because it is a simple problem that is still complex enough to demonstrate the capabilities of our framework.

A natural starting point for the framework was to create the structure of a sample graph, a 1-bit adder. It is easy to add too much information to each fact, and our final implementaion ended up with a much more minimalistic approach. Clingo is most effective with simple facts that encode a lot of information, marking one of the contributions our framework has to the field of both logic programming and neural networks.

For our example network we used neurons with three types of well established logic gates: AND, OR, and XOR. In Table 1 we show the definitions of these gates, and their truth tables. Using a combination of these and more gates, one is able to construct complex logical operations¹. Moreover, we be-

¹An elegant and simple proof of Clingo's Turing completeness falls out of the C-MCPN Framework.

lieve that the C-MCPN Framework has the potential to be used for a wide range of applications in the future, especially if its capabilities are further explored and developed.

AND			OR			XOR		
$g(\vec{x}) = x_1 \cdot x_2, \theta = 1$			$g(\vec{x}) = x_1 + x_2, \theta = 1$			$g(\vec{x}) = x_1 - x_2 , \theta = 1$		
Activation:						$f(\vec{x}) = \begin{cases} 0 & \text{if } g(\vec{x}) < \theta, \\ 1 & \text{if } g(\vec{x}) \geq \theta. \end{cases}$		
AND			OR			XOR		
x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$
T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	T
F	T	F	F	T	T	F	T	T
F	F	F	F	F	F	F	F	F

Table 1: McCulloch-Pitts Neuron Definitions and Truth Tables.

1.2 Constructing the Editor

Alongside the framework, we also built an editor for constructing and visualizing logical networks using the C-MCPN Framework. It is an entirely local desktop application which we felt would be more accessible to a wider range of users. The framework works well in a terminal, but having a live visual representation of the networks is a powerful tool for understanding and debugging them.

The editor is currently built in Python² using the DearPyGui library, allowing us to take advantage of its node-based editing capabilities. This meant we could keep the dependencies of the editor to a minimum, and focus on building the features we wanted.

Building the editor in this way also allowed us to easily integrate it with the C-MCPN Framework, as we could directly use the same two-layered data structures for both; one for the network architecture, and one for the values of each neuron. We also ensured that it was easy to add new gates to the editor. This was an important design choice, as it allows the editor to be easily extended in the future to support a wider range of applications. One more feature of note is the reorganization algorithm. We used a modified version of

²Python 3.8.5 is required (in part?) due to its native support for the tomli package.

the Sugiyama algorithm, taking just the layer assignment step and a simplified node positioning step [Sugiyama1981].

1.2.1 AI-Assisted Editing

Claude (Anthropic) was used as a programming assistant throughout development of the editor, helping with DearPyGui module integration, debugging, and some code generation. All design decisions, research direction, and domain-specific logic (ASP, gate definitions, network structure) were made by the authors. Every line generated by Claude was reviewed, edited, and approved by the authors before being added to the codebase, and it did not have direct access to the codebase at any point. All of the files Claude had input in are contained in the `app/` directory of the project, though not all files in this directory had input from Claude.

2 Implementation

2.1 C-MCPN Framework

From what we could find in the literature, the C-MCPN Framework is a novel approach to neural network representation within a logical programming paradigm like ASP. The networks themselves have relatively simple representations that can be leveraged to create complex operations. The framework uses three main program types:

- **Gate Definitions Program:** Files are in the `C-MCPN/gates/` directory.
- **Network Structure Program:** Files are in the `C-MCPN/networks/` directory.
- **Value Inference Program:** Files are in the `C-MCPN/base/` directory.

The files in the gate definitions program define the behavior of each gate, including the number of inputs it should have. Every default gate we included also has a special header used to send meta-information to the C-MCPN Editor. For example, the file defining the AND gate looks like this:

```
%% gate: AND
%% inputs: 2
%% outputs: 1

% type("TYPE")
type("AND").
```

```
% gate(TYPE, OUTPUT, INPUTS)
gate("AND", Out, In1, In2) :-
    values(In1), values(In2),
    Out = (In1 * In2).
```

Note the metadata in the header, and the two rules which guide this gate's definition. The first simply defines the gate, and the second performs an operation on its inputs. Then, the files in the network structure program define the neurons which exist and their connections. For example, the file defining the 1-bit adder includes the following lines:

```
% neuron(type, unique_id)
neuron("INPUT", "INPUT_1").
neuron("XOR", "XOR_1").

% edge(source_id, target_id)
edge("INPUT_1", "XOR_1").

% { edge(source_id, target_id) : neuron(type, target_id) } max_inputs.
{ edge(src, "XOR_1") : neuron("XOR", "XOR_1") } 2.

% val(neuron_id, value).
val("INPUT_1", 1).
```

Note the cardinality constraints which enforce the correct number of inputs for each neuron, as well as the fact which sets the value of the input neuron to 1. Finally, the value inference program performs two important roles. It activates each gate by allowing the edges between neurons to carry values, and it assigns each individual neuron a value based on its gate type. The following lines are from the value inference program, and show how this is implemented:

```
%% Value domain
values(0;1).

% Edges carry values from input to output
edge(In, Neuron_id, Val) :-
    edge(In, Neuron_id),
    neuron(_, Neuron_id),
    neuron(_, In),
    val(In, Val).
```

```

% 2 input gates
val(Neuron_id, Val) :-
    gate(Type, Val, Val1, Val2),
    neuron(Type, Neuron_id),
    edge(In1, Neuron_id, Val1),
    edge(In2, Neuron_id, Val2),
    In1 != In2.

%% Each neuron has one value
{ val(Neuron_id, Val) : values(Val) } 1 :- neuron(_, Neuron_id).

%% Show values
#show val/2.

```

Note the first fact which sets the domain of values to be binary and the constraint that ensures each neuron has exactly one value. Most importantly, the rule which carries the values along the edges is vital to the functioning of the network. Without it, the network gets "stuck" and is unable to infer what the values of the neurons should be. This is a key insight we had during development.

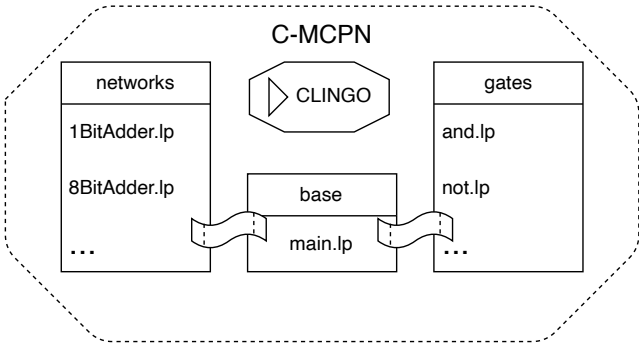


Figure 1: A visualization of the programs involved in the C-MCPN Framework.

2.2 C-MCPN Editor